Decuparea vectorilor

Prof. univ. dr. ing. Florica Moldoveanu

Curs *Elemente de Grafică pe Calculator* – UPB, Automatică şi Calculatoare
2018-2019

Decuparea primitivelor grafice

Decuparea: necesara pentru obiectele partial vizibile in imagine

Intr-un sistem grafic 2D:

— Decupare la frontiera unui dreptunghi din spatiul de afisare, avand laturile paralele cu cele ale sistemului de coordonate carteziene 2D (viewport)

— Decupare la frontiera unui poligon oarecare din spatiul de afisare

— Decuparea se poate efectua:

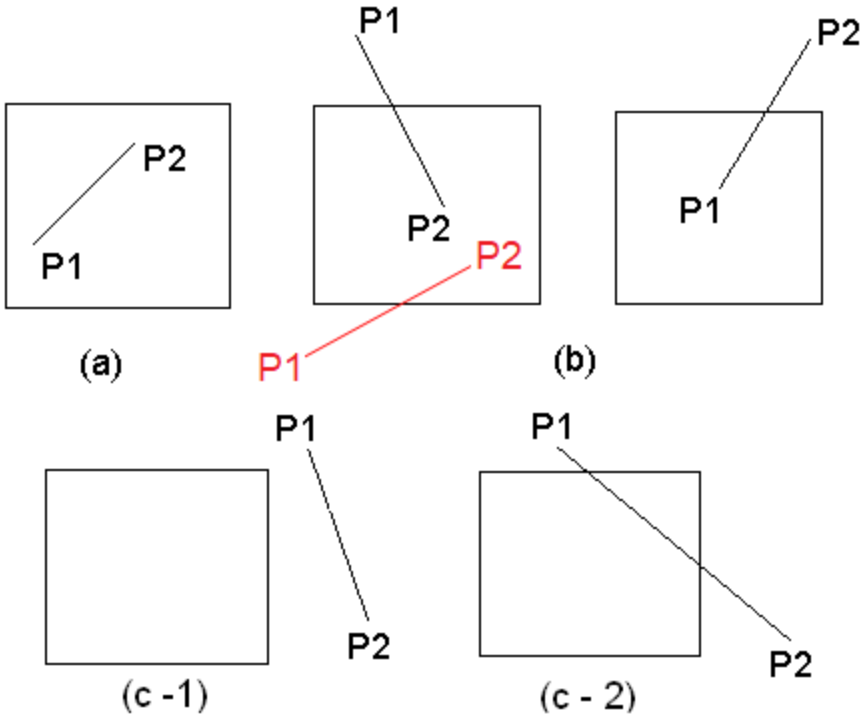
- Analitic, prin intersectia primitivelor grafice cu frontiera zonei de decupare (dreptunghi/poligon oarecare), inainte de rasterizare
- La momentul rasterizarii primitivelor, prin testul de apartenenta a fiecarui fragment la dreptunghiul/poligonul de decupare – de ex. pentru cercuri si elipse

— Pentru linii si poligoane, decuparea se efectueaza analitic (mai eficient).

Intr-un sistem grafic 3D bazat pe OpenGL:

— Decuparea se efectueaza la frontiera volumului vizual canonic, in coordonate de decupare (dupa transformarea de proiectie, inainte de transformarea in poarta de afisare).

Decuparea vectorilor 2D(1)



Pozitia unui vector fata de dreptunghiul de decupare

Exemplu: intersectie P1-P2 cu dreptunghiul neţinând cont de poziţiile punctelor P1, P2 faţă de dreptunghiul de decupare:

1. Calcul intersectie cu latura din stanga
2. Calcul intersectie cu latura de sus
3. Calcul intersectie cu latura din dreapta
4. Calcul intersectie cu latura de jos

1, 2, 3 – calcule inutile

Optimizari: evitarea calculelor de intersectie inutile

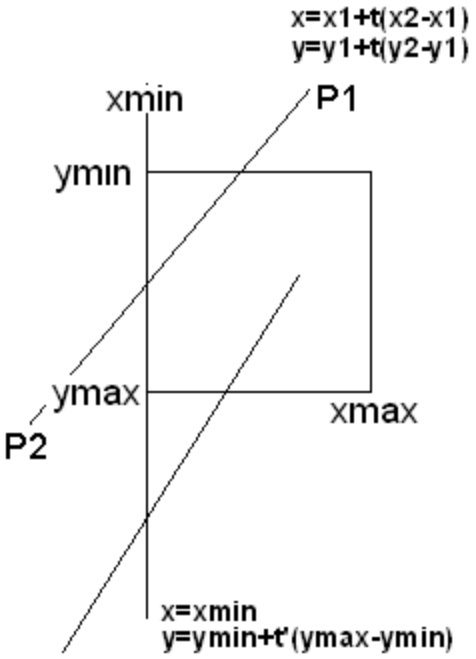
- Evitarea calculelor de intersectie in cazurile de tip (c-1) :

  - cum se diferentiaza cazurile (c-1) de cazurile (c-2) fara a efectua calcule de intersectie?

- Determinarea laturii/laturilor intersectate in cazurile de tipul (b) si (c-2), fara calcule de intersectie.

Decuparea vectorilor 2D(2)

Calculul intersectiilor cu laturile dreptunghiului de decupare



Intersectia segmentului P1-P2 cu latura x = xmin a dreptunghiului

Ec. parametrica a vectorului P1-P2:

P(t) = P1 + (P2-P1)*t 0<= t <=1 intre P1-P2

Punctul de intersectie cu dreapta x=xmin are abscisa xmin:

1) xmin=x1+t(x2-x1) → t=(xmin-x1)/(x2-x1)

2) este 0<=t<=1 ???

nu: punctul de intersectie se afla pe prelungirea segmentului

da: punctul de intersectie se afla pe segmentul P1-P2

- se calculeaza yi=y1+(xmin-x1)*(y2-y1)/(x2-x1)
- este ymin<= yi <= ymax ??

da: exista intersectie intre vectorul P1-P2 si latura x=xmin

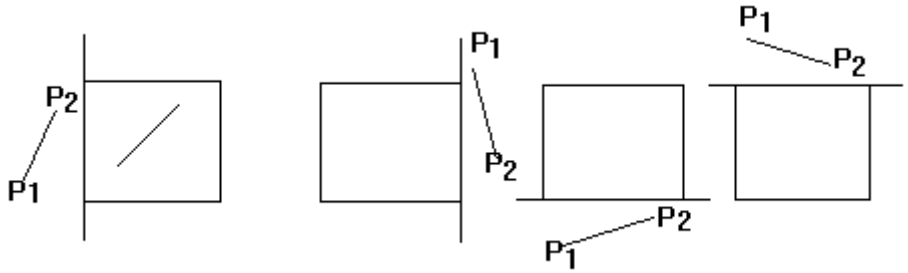
nu: punct de intersectie cu prelungirea laturii – calcul inutil

Algoritmul Cohen-Sutherland (1)

1001	1000	1010
0001	0000	0010
0101	0100	0110

- Punctele din plan sunt codificate prin 4 biti, b3b2b1b0, in functie de pozitia lor fata de dreptunghiul de decupare. De exemplu, se face conventia:
- b0=1: pentru punctele din stanga dreptunghiului,
- b1=1: dreapta, b2=1: sub dreptunghi, b3=1: deasupra dreptunghiului

Fie **cod1** codul binar al punctului P1 si **cod2** codul binar al punctului P2.



daca (cod1 ==0 && cod2==0) segment este acceptat trivial;

altfel

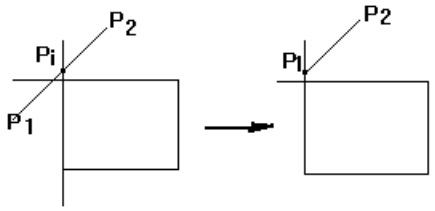
daca ((cod1 & cod2) != 0) segmental este rejectat trivial
(cele 2 coduri au acelasi bit =1)
P1 si P2 de aceeasi parte a dreptunghiului
→ segmentul nu intersecteaza dreptunghiul

altfel: se intersecteaza segmentul P1-P2 cu dreptunghiul de decupare

Algoritmul Cohen-Sutherland (2)

- Se alege latura de intersectie cu segmentul P1-P2 pe baza codului varfului P1, care trebuie sa fie situat in afara dreptunghiului de decupare.
- Daca cod1 = 0 → se interschimba P1 cu P2

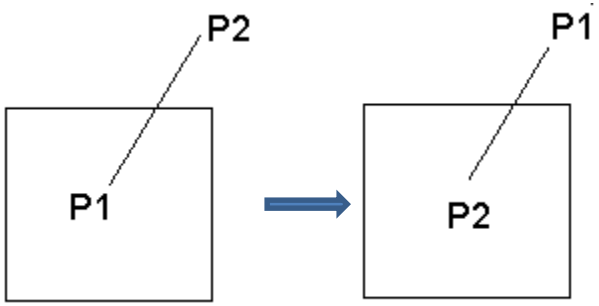
cod1 & 1 != 0
P1 in stanga dreptunghiului



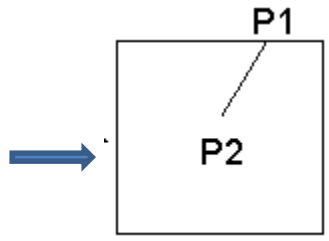
- Se calculeaza intersectia cu x=xmin
- Se muta P1 in punctul de intersectie
- In iteratia urmatoare cod1&cod2 != 0

  → segmentul este rejectat

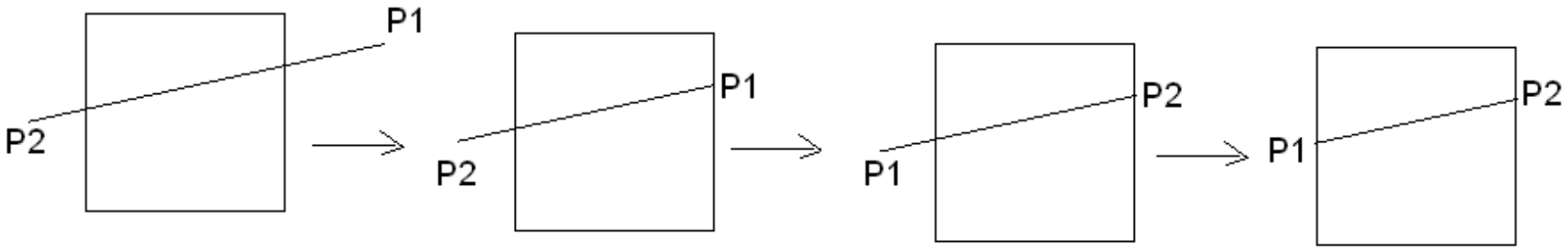
cod1 = 0



cod1 & 8 != 0
P1 deasupra dreptunghiului



Algoritmul Cohen-Sutherland (3)



Calculul punctelor de intersectie

$P(t) = P1 + (P2-P1)*t \rightarrow x(t) = x1 + t(x2-x1);\ y(t) = y1 + t(y2-y1)$

Intersectia cu latura situata pe dreapta y = ymax

ymax=y1+t(y2-y1) de unde, t=(ymax-y1)/(y2-y1)

x=x1+(ymax-y1)*(x2-x1)/(y2-y1) → **xi=x1+(ymax-y1)/m** m: panta segmentului P1-P2

Intersectia cu latura situata pe dreapta x = xmin

xmin=x1+t(x2-x1), deci t=(xmin-x1)/(x2-x1)

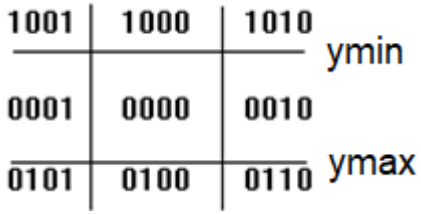
y=y1+(xmin-x1)*(y2-y1)/(x2-x1) → **yi=y1+(xmin-x1)*m**

Algoritmul Cohen-Sutherland (4)

```
int xmin, ymin, xmax, ymax;  // colturile dreptunghiului de decupare

// functia codif calculeaza codul binar al unui punct din plan, pe baza pozitiei sale fata de
     dreptunghiul de decupare

int codif(int x, int y)
{
  int cod =0;
  if(x < xmin)  cod =1;
  if(x > xmax) cod=2;
  if(y > ymax)  cod |=4;  //b2=1
  if(y < ymin) cod |=8;  //b3=1
  return  cod;
}
```



Algoritmul Cohen-Sutherland (5)

```
int Cohen_Suth(int x1i, int y1i, int x2i, int y2i, int* x1d, int* y1d, int* x2d, int* y2d)
{ // primeste coord. capetelor vectorului de decupat si intoarce coord. partii din vector
  // inclusa in dreptunghi, daca exista
  int acceptat,rejectat,vertical;
  float m, x1=x1i, y1=y1i, x2=x2i, y2=y2i;
  int cod1, cod2, t;
  acceptat=rejectat=vertical=0;
  if (x1i != x2i)  //vectorul nu este vertical
      m=(y2-y1)/(x2-x1);
  else
      vertical=1;
 do
   { cod1=codif(x1,y1);  cod2=codif(x2,y2);
     if(cod1==0 && cod2==0) { acceptat=1; break;}
     if((cod1 & cod2) !=0 )  { rejectat=1; break;}
```

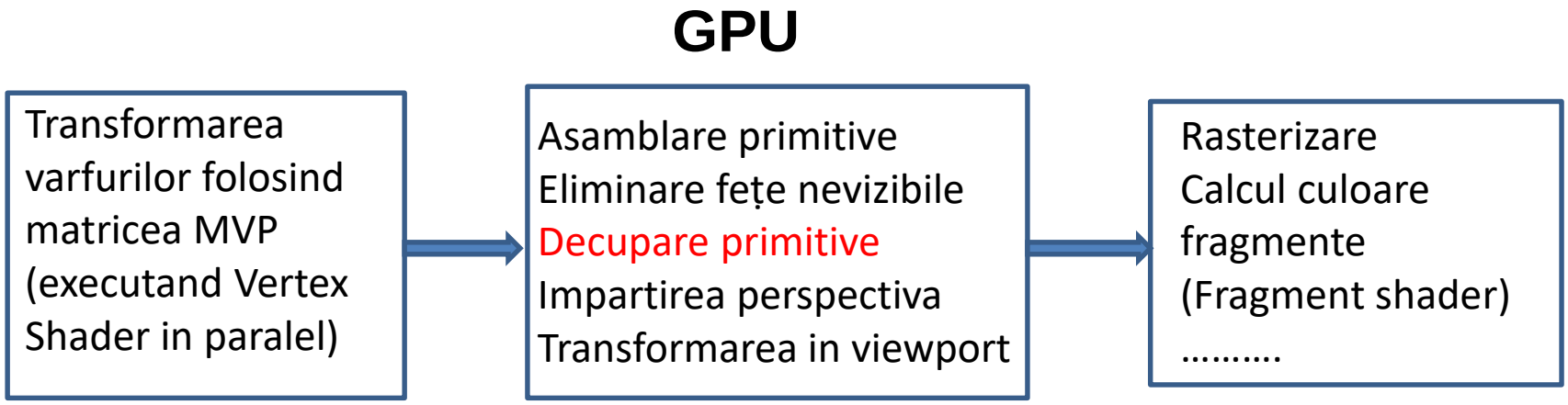
Algoritmul Cohen-Sutherland (6)

```
if(cod1==0) // P1 este in interiorul dreptunghiului de decupare
    {t=x1;x1=x2;x2=t; t=y1;y1=y2;y2=t; cod1=cod2;} //interschimbare capete
if(cod1 & 1)  //P1 in stanga dreptunghiului
     { y1+=(xmin-x1)*m;  x1=xmin;}  // punctul de intersectie devine P1 pentru iteratia urmatoare
  else
   if(cod1 & 2) //P1 in dreapta dreptunghiului
     { y1+=(xmax-x1)*m;  x1=xmax; }
      else
     if(cod1 & 4) // P1 sub dreptunghi
       { if (!vertical)   x1+=(ymax-y1)/m;
         y1=ymax;
       }
     else // P1 deasupra dreptunghiului
       {if (!vertical) x1+=(ymin-y1)/m;
        y1=ymin;
       }
  }   while ( acceptat==rejectat); // cat timp vectorul nu este acceptat sau rejectat
```

Algoritmul Cohen-Sutherland (7)

```
 if(acceptat)
  {*x1d=x1;*y1d=y1;*x2d=x2;*y2d=y2;
   return 1;
  }
  return 0;
 }
}
```

Decuparea în banda grafica (1)



- Decuparea se efectueaza asupra primitivelor 3D, la frontiera volumului vizual canonic
- Din decupare pot rezulta noi varfuri
- Varfurile primitivelor pot avea asociate atribute: culoare, normala, coordonate textura
- Pentru varfurile rezultate din decupare se calculeaza atribute prin interpolarea atributelor asociate varfurilor, folosind valoarea variabilei parametrice (t) în punctul de decupare.

Decuparea în banda grafica (2)

Transformarea de proiectie:

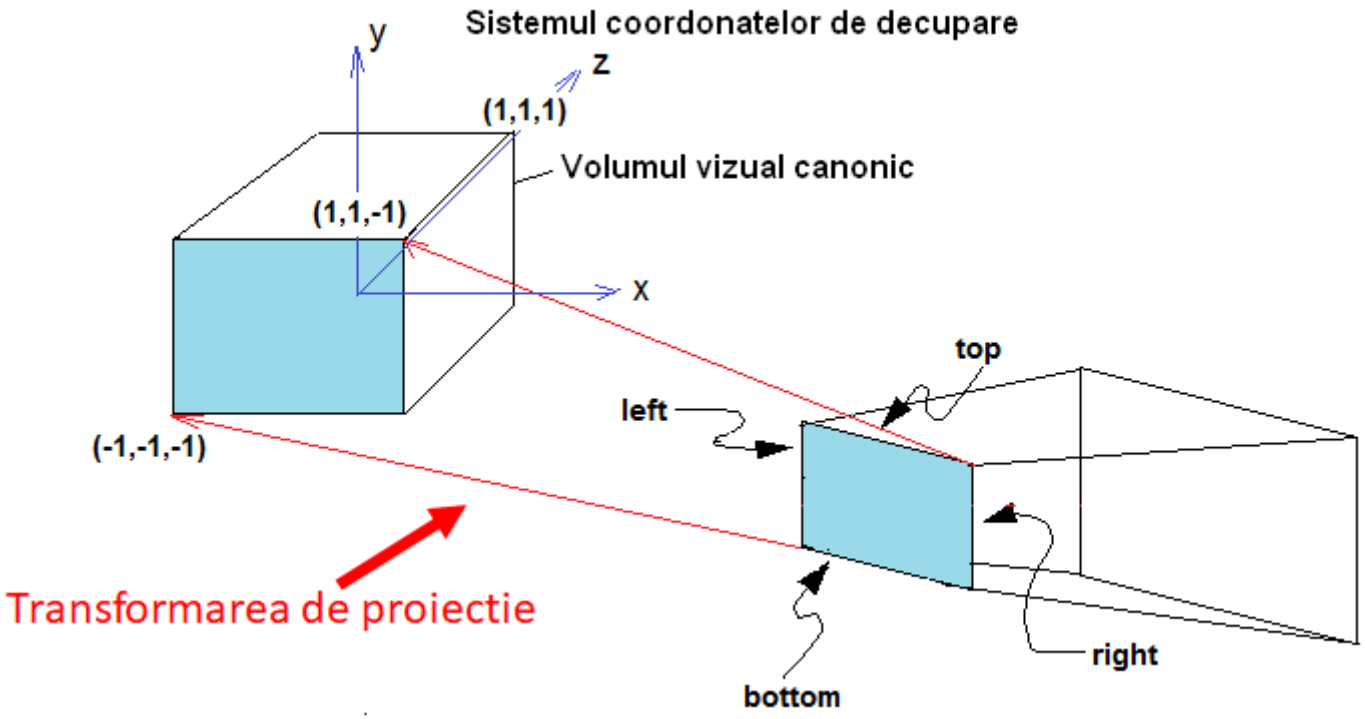
$[xc\ yc\ zc\ w]^T = P \bullet [xe\ ye\ ze\ we]^T$

(clip coordinates) ← (eye coordinates)

Coordonate omogene → împartirea perspectiva → coordonate carteziene:

xd = xc/w , yd = yc/w , zd= zc/w, w = -ze

(xd, yd, zd): coordonate dispozitiv normalizate (**N**ormalized **D**evice **C**oordinates)



Decuparea în banda grafica (3)

Decuparea: înainte sau dupa împarţirea perspectivă?

1. Inainte: decupare in coordonate omogene
2. Dupa: decupare in coordonate carteziene (NDC)

- Problema: împartirea perspectiva

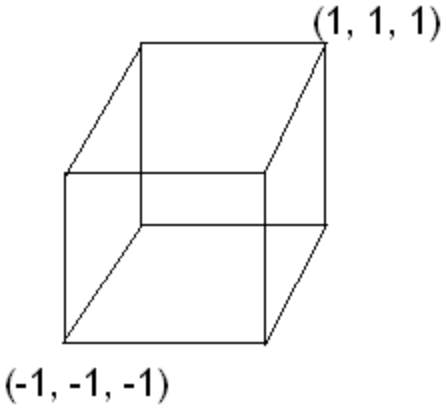
  xd = xc/w , yd = yc/w , zd= zc/w

  w=- ze = 0 pentru proiectia perspectiva daca punctul transformat se afla in acelasi plan cu camera (centrul de proiectie): ze = 0

Problema nu apare in cazul 1, caci varfurile / primitivele din planul camerei sunt eliminate la decupare, ele nefiind incluse in volumul vizual (near>0).

In cazul 2, la conversia in NDC trebuie tratat cazul special w=0.

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(1) (decuparea în coordonate carteziene)



Faţă de volumul vizual canonic, un punct din spatiu se poate afla: in interior, in stanga, in dreapta, sub volum, deasupra volumului, in fata, in spate:

→pentru codificarea pozitiei unui punct din spatiu fata de volumul vizual canonic sunt necesari 6 biti.

De ex. se poate face conventia:

b0 = 1 pentru puncte (x,y,z) cu x < -1

b1 = 1 pentru puncte (x,y,z) cu x > 1

b2 = 1 pentru puncte (x,y,z) cu y < -1

b3 = 1 pentru puncte (x,y,z) cu y > 1

b4 = 1 pentru puncte (x,y,z) cu z < -1

b5 = 1 pentru puncte (x,y,z) cu z > 1

daca: cod(P1) ==0 && cod(P2)==0
    segment acceptat trivial

daca (cod(P1) & cod(P2)) != 0
    segment rejectat trivial
altfel
    se intersecteaza segmentul cu volumul

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(2)

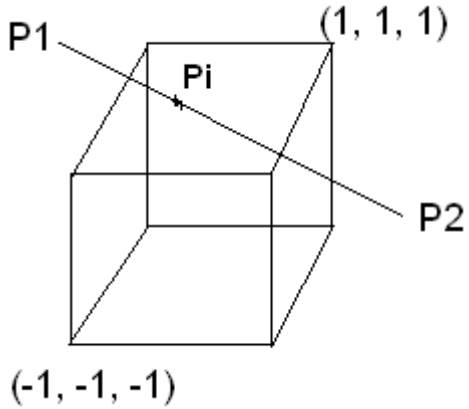
Intersectiile segmentului cu volumul vizual

Ec. parametrice ale segmentului

$x = x1 + t(x2 – x1)$

$y = y1 + t(y2 –y1)$

$z = z1 + t(z2 – z1)$



daca (cod(P1) & 8 !=0) //P1 deasupra volumului

Se efectueaza intersectia cu planul y= 1:

$1 = y1 + t(y2 – y1) \rightarrow tinters = (1-y1)/(y2-y1)$

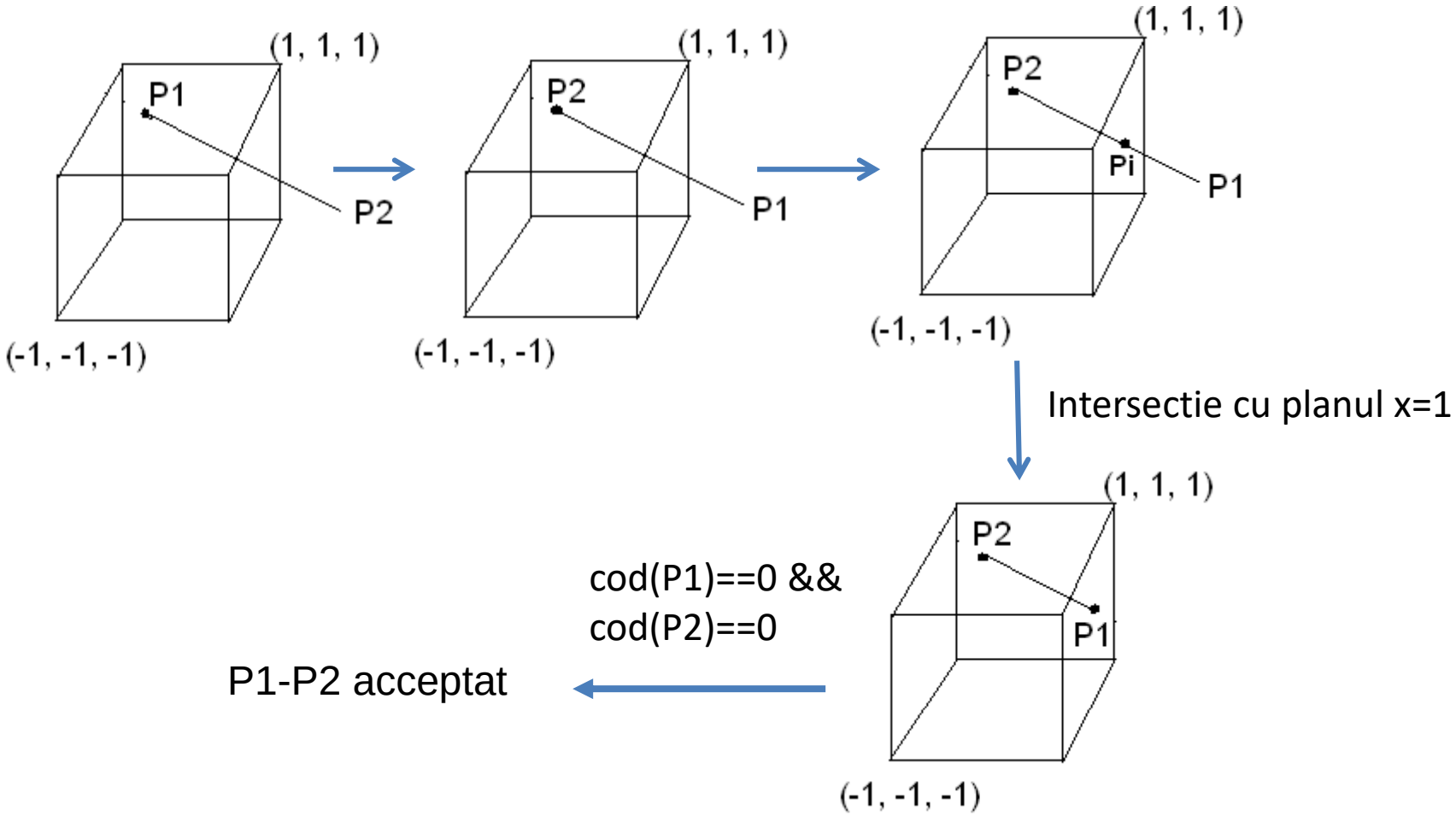
Calculeaza punctul de intersectie si muta P1 in punctul de intersectie

$x1 = x1 + tinters(x2 – x1)$

$y1 = 1;$

$z1 = z1 + tinters(z2 – z1)$

Generalizarea algoritmului Cohen-Sutherland pentru decuparea vectorilor 3D(3)



Algoritmul Liang-Barsky (1)

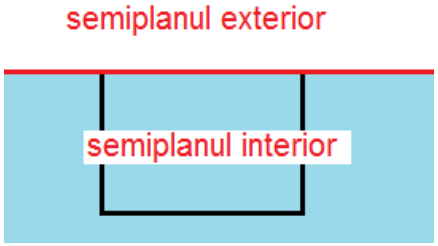
- Cyrus si Beck (1978): **algoritm de decupare parametrica**
  - a unui vector 2D la frontiera unui dreptunghi sau a unui poligon convex oarecare;
  - a unui vector 3D la frontiera unui poliedru convex oarecare
  - mai eficient decat algoritmul Cohen-Sutherland atunci cand majoritatea liniilor intersecteaza dreptunghiul/poliedrul de decupare.
- Liang si Barsky (1984): algoritm de decupare parametrica mai rapid decat algoritmul Cyrus-Beck pentru cazurile in care zona de decupare este un dreptunghi sau un paralelipiped aliniat cu axele sistemului de coordonate carteziene 2D/3D.

Decupare parametrica: se determina punctele de intersectie dintre vector si frontiera zonei de decupare pe baza valorilor variabilei parametrice (t) in punctele de intersectie.

Algoritmul Liang-Barsky (2)

- Fiecare latura a zonei de decupare împarte planul în 2 semiplane:

- semiplanul interior – care contine zona de decupare
- semiplanul exterior



- Fiind dat un segment P1-P2, se va afisa numai acea parte a sa care este conţinută în semiplanul interior faţă de fiecare latură a zonei de decupare.
- Pozitia unui punct fata de o latura a zonei de decupare se determina folosind normala la latura respectiva, orientata înspre exteriorul zonei de decupare.
- Fie P(t) = P1 + (P2 – P1)*t ecuatia vectorului decupat

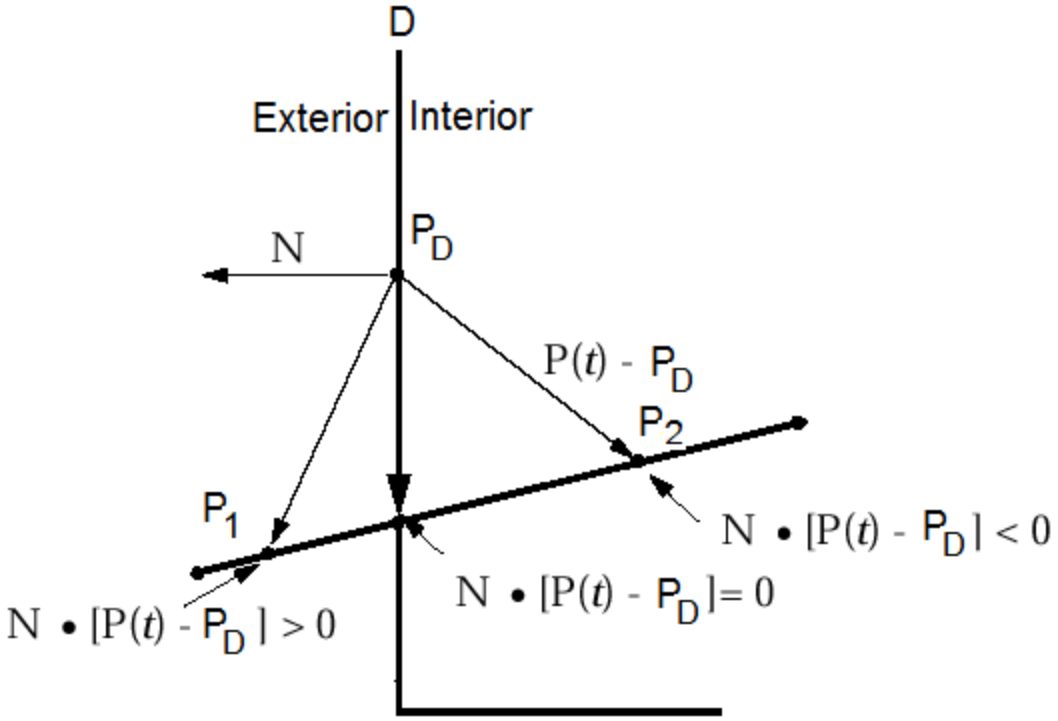
  t=0 → P1 si t=1 → P2

  D – dreapta pe care se afla latura zonei de decupare

  P_D - un punct de pe dreapta D

Algoritmul Liang-Barsky (3)

- Se poate determina pozitia unui punct P(t), de pe vectorul [P2 - P1], fata de zona de decupare examinand valoarea produsului scalar dintre N si vectorul [P(t) - P_D], unde N este normala la latura D a zonei de decupare.



Produsul scalar a 2 vectori:

V1 • V2 = |V1|*|V2|*cos(u)

0<=u<90 → V1 • V2 >0

u=90 → V1 • V2 = 0

90< u <=180 → V1 • V2 <0

Algoritmul Liang-Barsky (4)

- Valoarea lui t în punctul de intersectie cu dreapta D

  $N \bullet [P(t) - P_D] = N \bullet [P1 + (P2 - P1)*t - P_D] = 0$ (vectori perpendiculari)

  $N \bullet [P1 - P_D] + N \bullet [(P2 - P1)*t] = 0$

Rezulta:

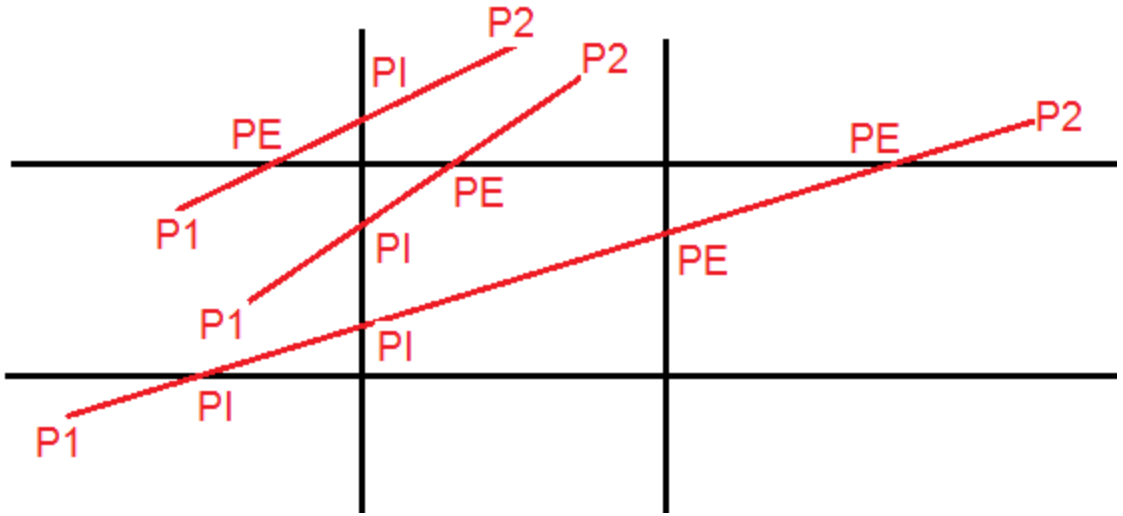
$t = - (N \bullet [P1 - P_D]) / (N \bullet [P2 - P1])$

- Daca $N \bullet [P2 - P1] = 0$ → (P1->P2) este paralel cu latura D → nu intersecteaza D.
- Altfel, se calculeaza valorile t1, t2, t3, t4, corespunzatoare intersectiilor dintre vectorul (P1->P2) si dreptele pe care se afla laturile dreptunghiului de decupare.
- Se retin numai valorile 0 <= ti <= 1 (punctul de intersectie sa fie intre P1 si P2)
- Se verifica pentru fiecare valoare ti retinuta daca punctul de intersectie se afla pe latura dreptunghiului de decupare (si nu pe prelungirea sa)

Algoritmul Liang-Barsky (5)

- Punctul de intersectie dintre vectorul (P1->P2) si o latură a dreptunghiului de decupare este clasificat in:

- Punct potential de intrare in dreptunghiul de decupare, PI, daca N • [P2 – P1] < 0
- Punct potential de iesire din dreptunghiul de decupare, PE, daca N • [P2 – P1] > 0



PI – intrare in semiplanul interior al laturii
PE – iesire din semiplanul interior al laturii

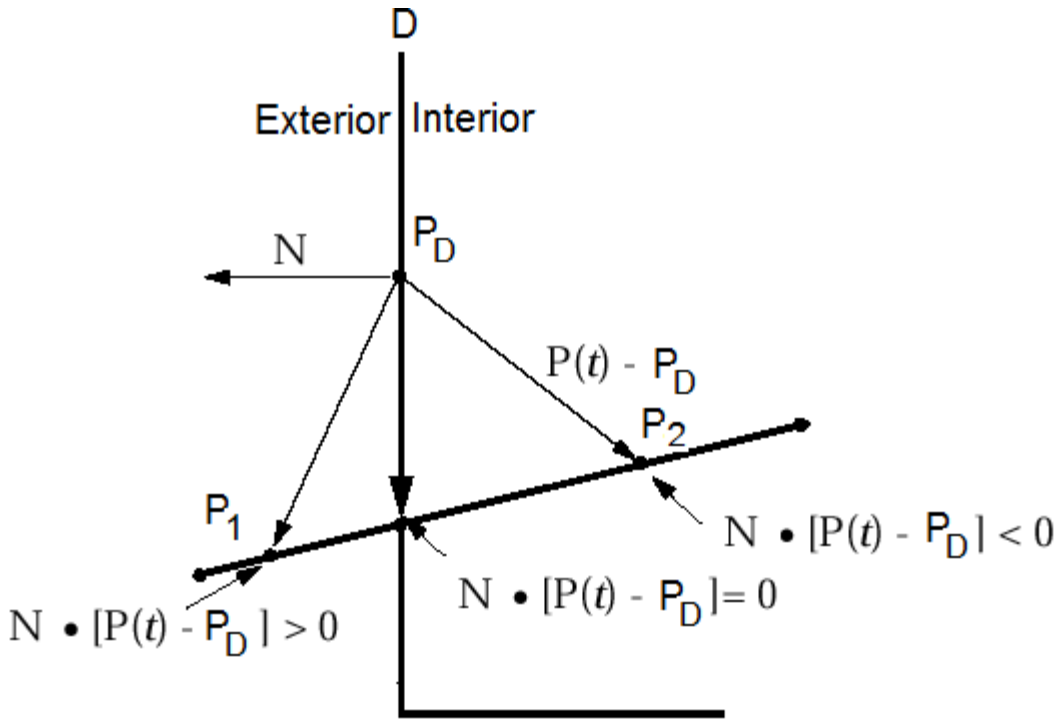
- Partea din vectorul (P1->P2) inclusa in dreptunghiul de decupare este segmentul (P(timax) -> P(temin)), cu timax >= 0 si temin <=1 si timax < temin
- Daca timax > temin → vectorul P1->P2 nu intersecteaza dreptunghiul de decupare.

Algoritmul Liang-Barsky (6)

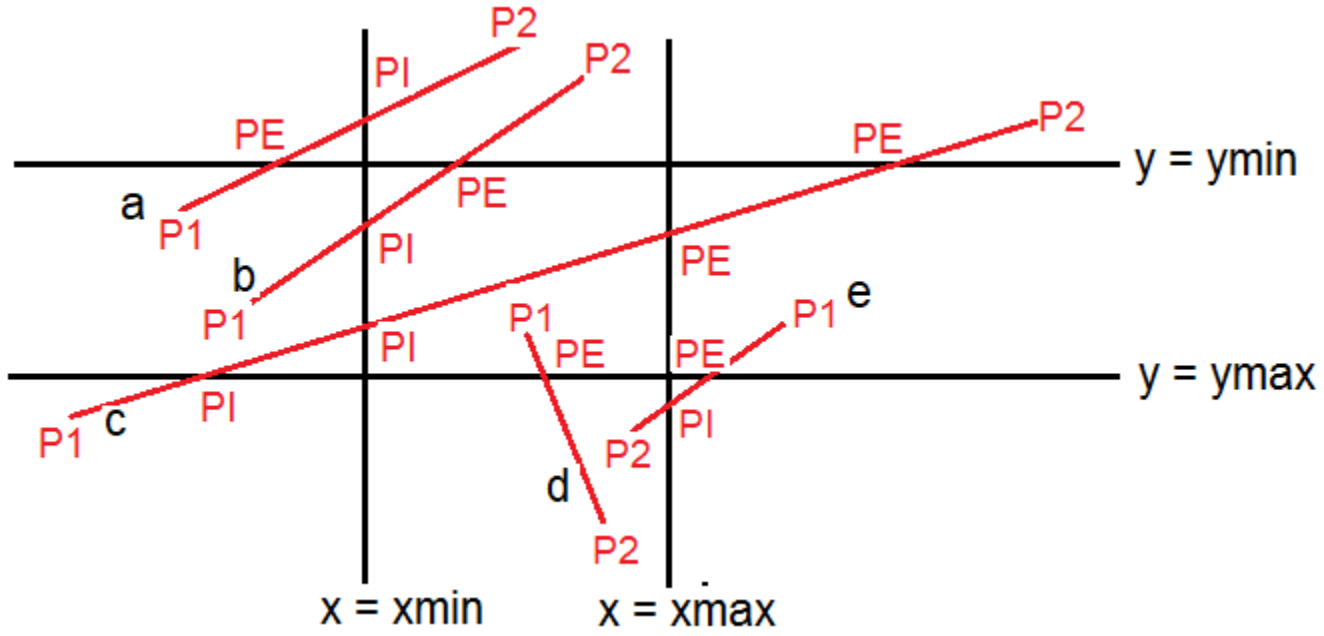
Calculul lui t pentru cele 4 laturi ale dreptunghiului de decupare:

$t = - (N \bullet [P1 – PD]) / (N \bullet [P2 – P1])$

1. D: x= xmin
   N =[-1,0]; N • [P2 – P1] = -(x2-x1) = -dx
   P_D = (xmin, y)
   P1 - P_D = [x1 – xmin, y1 –y]
   t = (xmin – x1)/dx
2. D: x= xmax
   N =[1,0]; N • [P2 – P1] = dx
   P_D = (xmax, y)
   P1 - P_D = [x1 – xmax, y1 –y]
   t = (xmax – x1)/dx
3. D: y= ymin
   N =[0,-1]; N • [P2 – P1] = -dy
   P_D = (x, ymin)
   P1 - P_D = [x1 – x, y1 –ymin]
   t = (ymin – y1)/dy
4. D: y = ymax
   analog cu D: y = ymin



Algoritmul Liang-Barsky (7)



Cazul (a)

- se calculeaza t in intersectia cu x=xmin: ti1; timax = ti1
- se calculeaza t in intersectia cu y = ymin: te1; temin = te1

  timax > temin → segmentul P1->P2 este rejectat trivial

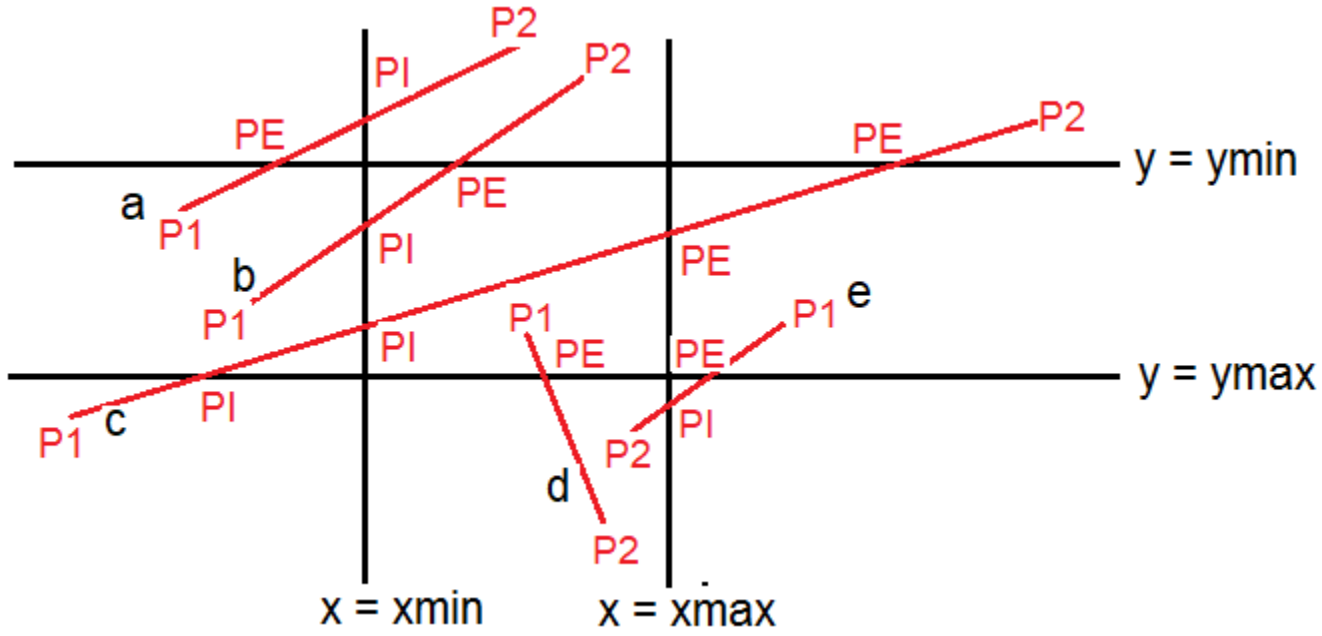
Cazul (b)

- se calculeaza t in intersectia cu x=xmin: ti1; timax = ti1
- se calculeaza t in intersectia cu y = ymin: te1; temin = te1

  timax < temin → se continua calculul lui t in intersectiile cu celelalte laturi
- in intersectia cu y=ymax t<0 iar in intersectia cu x=xmax t>1

  timax<temin →se intoarce segmentul P(timax)->P(temin)

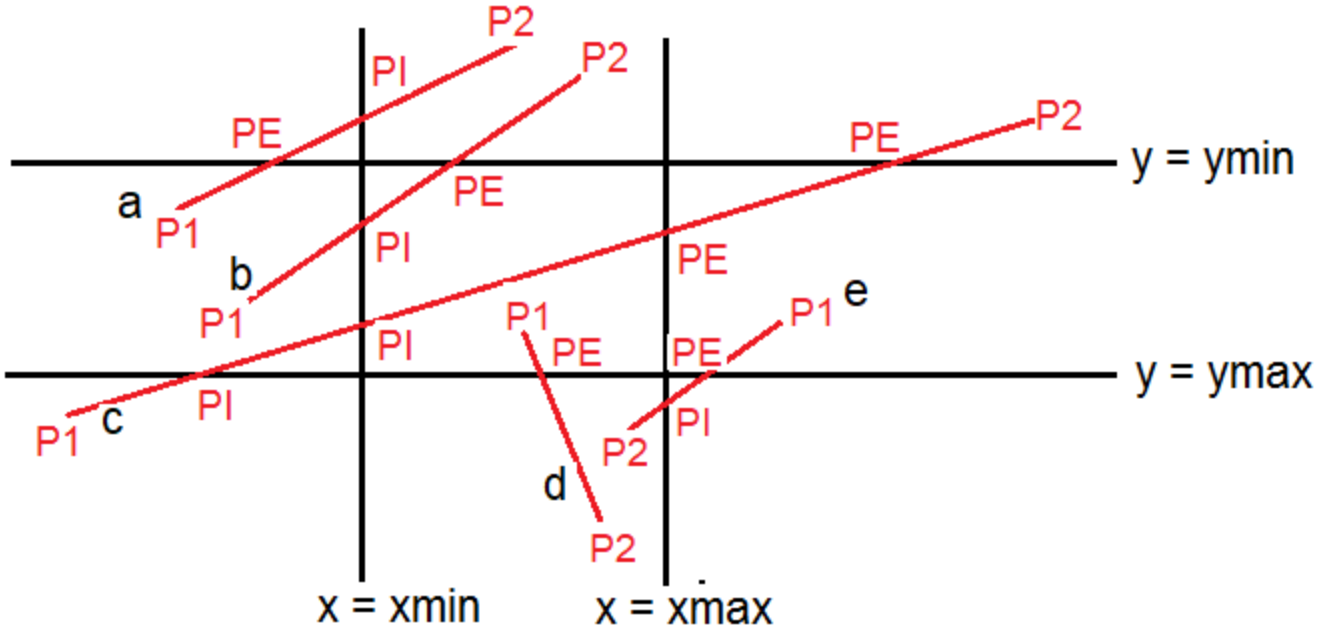
Algoritmul Liang-Barsky (7)



Cazul (c)

- se calculeaza t in intersectia cu x=xmin: ti1; timax = ti1
- se calculeaza t in intersectia cu y = ymin: te1; temin = te1
  timax < temin → se continua calculul lui t in intersectiile cu celelalte laturi
- se calculeaza t in intersectia cu x = xmax: te2; temin = te2
  timax < temin → se continua
- se calculeaza t in intersectia cu y = ymax: ti2; timax = ti1
  timax < temin → se intoarce segmentul P(timax)->P(temin)

Algoritmul Liang-Barsky (7)



Cazul (d)

- se calculeaza t in intersectia cu x=xmin: t<0, nu se retine
- se calculeaza t in intersectia cu y = ymin: t<0, nu se retine
- se calculeaza t in intersectia cu x = xmax: t>1, nu se retine
- se calculeaza t in intersectia cu y = ymax: te1; temin=te1
  → se intoarce segmentul P1->P(temin)

Cazul (e)

- se calculeaza t in intersectia cu x=xmin: t>1, nu se retine
- se calculeaza t in intersectia cu y = ymin: t<0, nu se retine
- se calculeaza t in intersectia cu x = xmax: ti1; timax = ti1
- se calculeaza t in intersectia cu y = ymax: te1; temin=te1
  timax>temin → segmentul P1->P2 este rejectat

Decuparea vectorilor - concluzii

- Algoritmul Cohen – Sutherland este eficient daca:

  - se folosesc operatii foarte simple pentru codificarea capetelor vectorilor si testarea lor

  - majoritatea vectorilor sunt acceptati sau rejectati trivial

- Decuparea parametrica (Algoritmul Liang- Barsky) este avantajoasa atunci cand majoritatea vectorilor traverseaza zona de decupare
- Ca si algoritmul Cohen-Sutherland, se generalizeaza usor pentru decuparea vectorilor 3D la frontiera volumului vizual canonic.